

Chapitre 1 – Android - Outils de développement

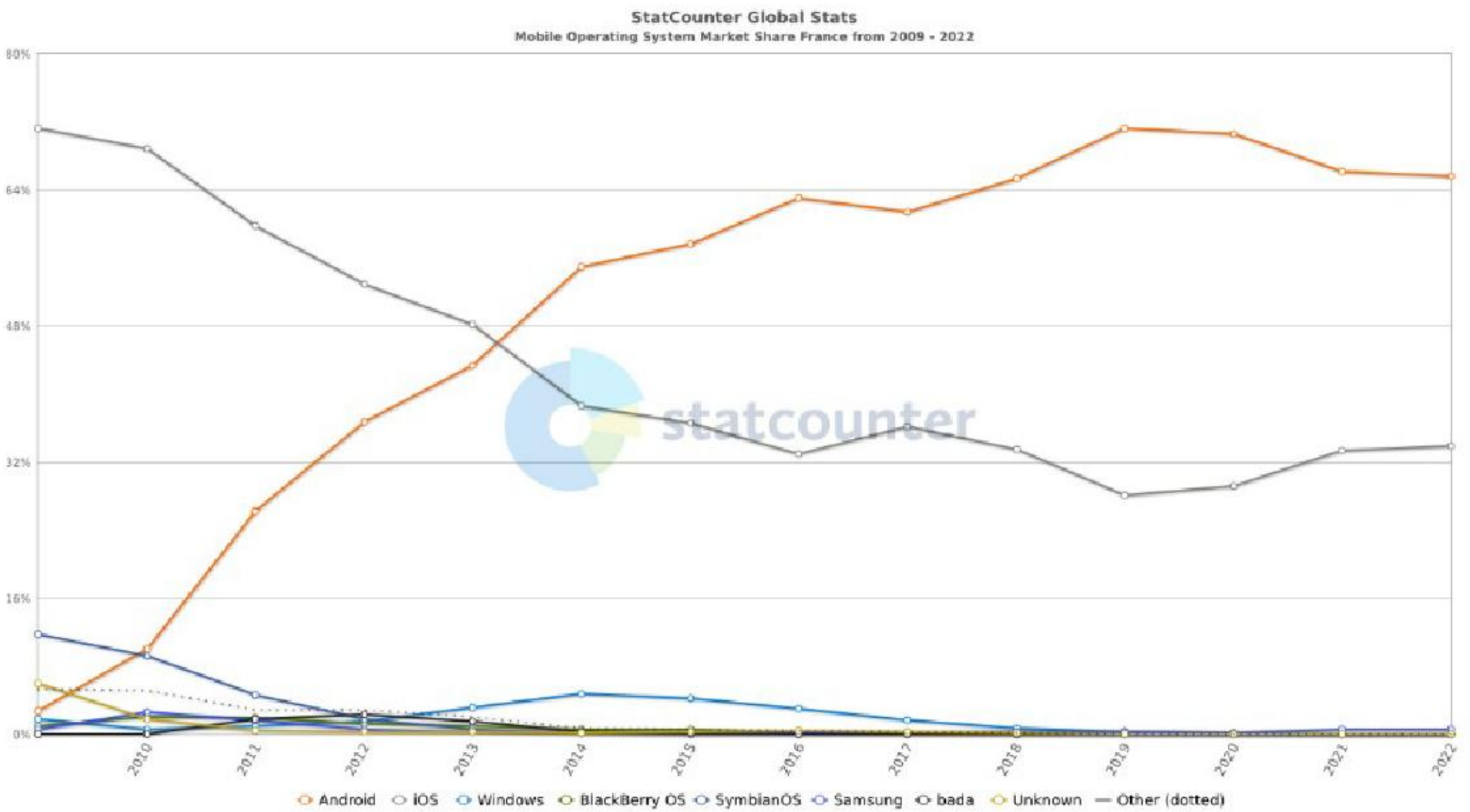
Sommaire

I.	Introduction.....	1
II.	Le JDK	2
III.	Le SDK Android	2
IV.	Choisir un EDI	3
V.	Premier projet Android	4
VI.	Adapter la VM.....	4
VII.	Organisation d'un projet Android.....	4

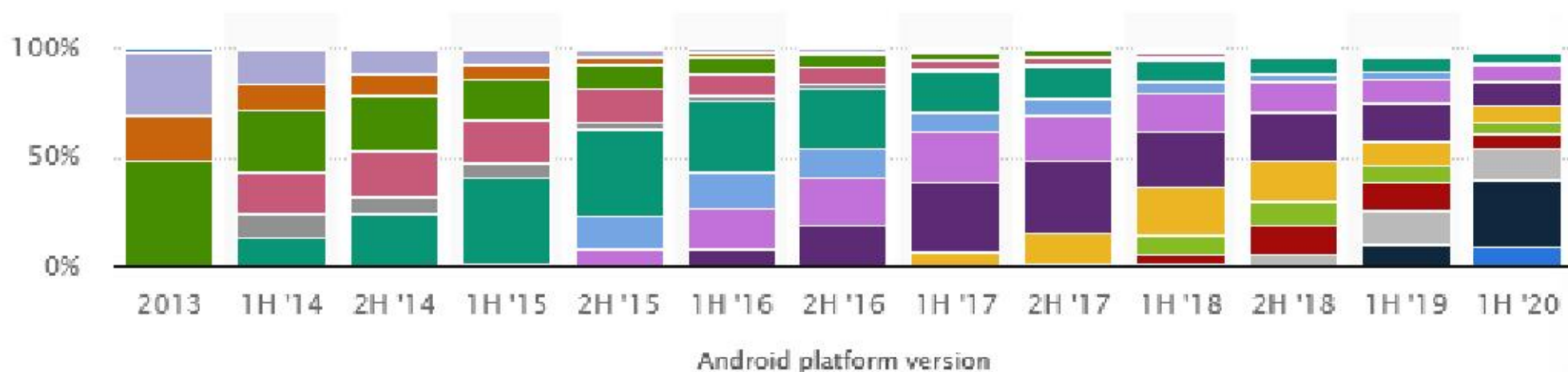
I. Introduction

Android est un système d'exploitation très récent (sortie de la version 1.0 en Septembre 2008). Le développement sur *Smartphone* et *Tablette* est pourtant aujourd'hui largement dominé par ce système (>72%) :

Evolution des parts de marché par OS de 2007 à 2017 :



Plusieurs versions d'Android co-existent :



Développer pour mobile et tablette peut signifier :

- Réaliser des pages web adaptées à ces formats particuliers,
- Ecrire des applications dédiées, téléchargeables et exécutables seulement par ces mobiles et tablettes.

Le premier point revient à détecter le mobile, puis à proposer :

- Soit un site spécifique pour ce média. Exemple pour le site *www.monsite.fr*, créer *m.monsite.fr*,
- Soit un site web adaptatif (« responsive web design », voir *chapitre 10 Framework*).

Le deuxième point (applications dédiées) est plus ambitieux et intéressant. C'est celui qu'on étudiera ici.

II. Le JDK

1. Objectif

Le système Android repose sur une machine virtuelle (JVM) exécutant du code Java. Pour développer en Java, il faut donc le kit de développement Java (*Java Development Kit* soit JDK).

2. Installation

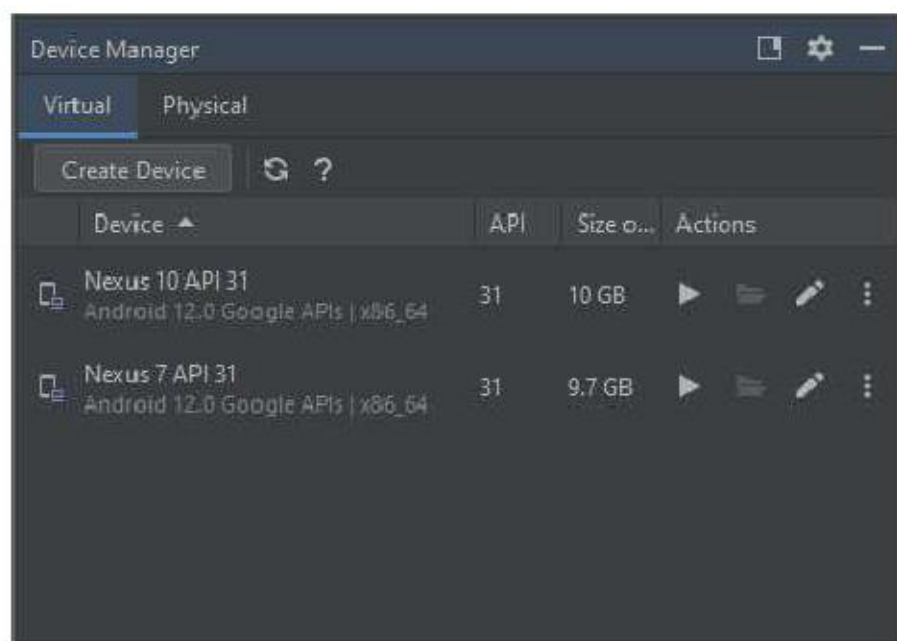
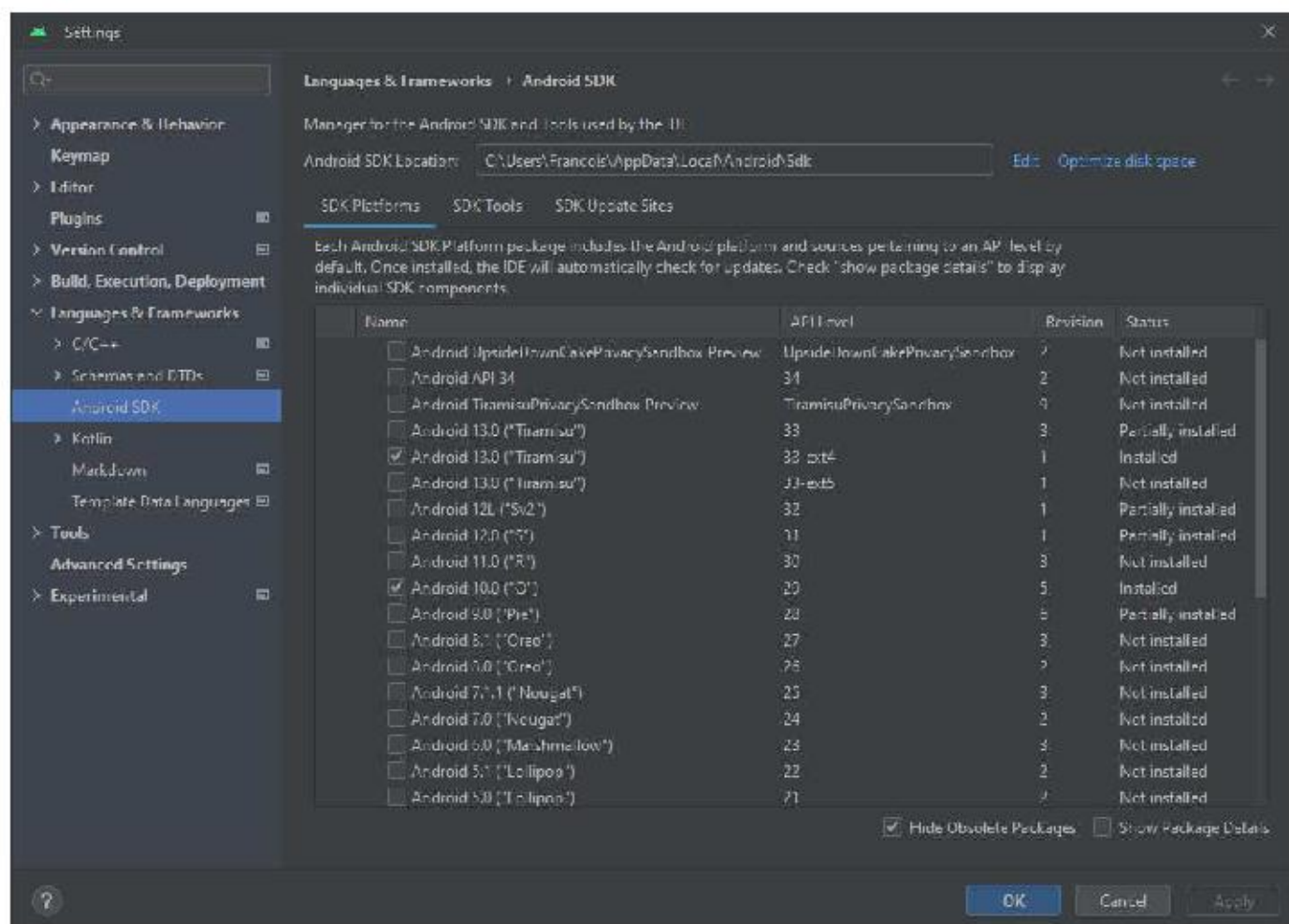
L'installation est aujourd'hui intégrée à *Android Studio*

III. Le SDK Android

1. Objectif

Les outils de développement spécifiques à Android (et non à Java) sont regroupés dans le SDK Android (SDK=Software Development Kit). Ils sont indispensables car ils permettent :

- De gérer le téléchargement des librairies avec le *SDK Manager*,
- De piloter un appareil virtuel avec l'*AVD Manager* (AVD=Android Virtual Device).



Les librairies sont associées à des versions d'Android. La version *Tiramisu*, correspond à la 33^{ème} évolution. On parle techniquement d'une API Level 33 (API=Application Programming Interface).

2. Installation

L'installation est aujourd'hui intégrée à *Android Studio*

IV. Choisir un EDI

1. Des EDI similaires

Plusieurs EDI (Environnement de Développement Intégré) existent pour le développement Android : *Netbeans*, *Eclipse* et *IntelliJ Idea*. Dernier arrivé : *Android Studio*, qui est une version spécifique d'*IntelliJ Idea*.

2. Installation d'Android Studio

Rechercher *Android Studio Download*. Choisir le site *developer.android.com*

3. Paramétrage d'Android Studio

Au premier lancement,

- Ne pas importer d'ancienne config ;
- Pas d'envoi d'info à Google ;
- Assistant - Bienvenue : *Suivant* ;
- Installation : *Standard* ; *Suivant* ;

- Thème : Au choix ; *Suivant* ;
- Synthèse : *Terminer* ;
- Téléchargement très important ...

V. Premier projet Android

Deux langages : *Kotlin* (nouveau) et *Java* (la référence). Privilégier *Java* (incontournable).

Depuis l'accueil d'Android Studio, choisir *New project*. Dans *Phone and tablet*, choisir *Empty Views Activity*, *Next*. Observer le paramétrage. *Finish*.

Laisser l'IDE travailler...

Lancer => Erreur (No target device found). Localiser *Target Device*, lancer *Device Manager*.

Créer un mobile virtuel : *Create device*, *Phone*, *480x800* ou *768x1280*, *Next*,

Lui affecter une version d'android : *Nougat*, *Oreo* ou *Pie*, *Next*,

Nommer et orientation portrait, *Finish*.

Attendre...

Relancer le projet. La VM est lancée car définie dans *Target Device*.

Première compilation toujours longue.

Hello world apparaît !

VI. Adapter la VM

Selon la version d'Android, légère différence... Tirer le panneau du haut, roue crantée (config), System, Languages & Input, Ajouter Français et retirer Anglais

Clavier physique : adapter à Français (azerty)

Pour fermer une application, cliquer sur le carré, balancer à droite ou à gauche.

Prendre le navigateur (chrome), vérifier :

- Le clavier virtuel Azerty
- La saisie depuis le PC (notamment le pavé numérique)
- L'accès à Internet depuis la VM

VII. Organisation d'un projet Android

Dans l'arborescence *App*, trois notions :

- Le fichier **manifest** : Point d'entrée de l'application – Contient ses « écrans », les fonctionnalités nécessaires (GPS, caméra, ...), le nom et l'icône de l'application essentiellement.
- Le dossier **java** : 3 dossiers qui semblent similaires. Seul le premier, qui contient *MainActivity*, abrite le code *java* de l'application.
- Le dossier **res** : toutes les ressources (images, design d'écrans, ...) de l'application.

Dans l'arborescence *Gradle Scripts*, seul le *build.gradle.kts* (*Module : app*) est à modifier éventuellement. Il reprend le *namespace* (nom du projet, sera utilisé sur le store), la version d'Android minimale requise pour l'app, et les bibliothèques intégrées au projet (par exemple création ou reconnaissance de QR Code, création de documents PDF, ...)

1. Fichier Manifest

Repérer le nœud *application* qui contient *activity* qui est le point d'entrée grâce à *intent-filter*.

Repérer aussi les @xxx qui font référence à des dossiers de *res* (ressources) :

- @xml pour res/xml,
- @mipmap pour res/mipmap
- @string pour res/values/strings.xml
- @style pour res/values/themes/themes.xml

Dans *activity*, repérer le lien avec le code java : *android:name=".MainActivity"*

2. Code Java

Développer l'arborescence *app, java, com.example.helloworld* (le premier des trois), *MainActivity*. Double-cliquer sur *MainActivity*. Quelques lignes pour :

- Indiquer le package (namespace) : *com.example.helloworld*
- Utiliser des bibliothèques : *import ...*
- Définir une activité (un écran/fenêtre) *MainActivity* comme classe héritée d'*AppCompatActivity* (écran de base)
- Définir la méthode *OnCreate* comme surcharge (ré-écriture) de celle d'*AppCompatActivity*. Elle appelle la méthode *OnCreate* de la classe parent, l'ancienne version donc... (c'est la fonction de *super*), puis définit la partie visible (« vue ») avec *R.layout.activity_main*. R fait référence au dossier *res*, le . remplace le /

Studio permet de souligner les fautes d'orthographe, ... mais seul le dictionnaire anglais est présent. Donc mieux vaut désactiver cette option : Activer *File, Settings*, puis dans *Editor*, cliquer sur *Spelling*, puis sur *Configure Spelling inspection*. Désactiver alors l'option *Typo de Proofreading*. Ok.

La compilation produit un fichier *application.apk* (*Android Packet Files*) qui contient toute l'application. Normalement, ce fichier est déposé dans le *Play Store* d'où il est téléchargé puis exécuté.

3. Interfaces utilisateur

Dans le dossier *res/layout*, on trouve la/les interfaces utilisateur : *activity_main.xml*

Noter l'absence de majuscule, et la règle de nommage différente de celle du code java.

Pour renommer un fichier, utiliser *Refactor, Rename* pour que les modifications liées se fassent.

Double-cliquer sur *activity_main.xml* :

- Contrôle+roulette permet de zoomer facilement,
- Repérer la palette, l'arborescence d'objets (*Component Tree*), les propriétés (*Attributes*) définies, puis courantes, enfin toutes (par ordre alphabétique)
- Basculer entre XML (code) et Design

Modifier le texte en *Hello les SIO* et lancer.

4. Quel API Level choisir ?

Cela dépend des fonctionnalités utilisées dans le projet, mais la règle est le plus bas possible, pour avoir une cible la plus large possible ...

Evolution de la répartition des versions des systèmes Android :

